



UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

DIPARTIMENTO DI INGEGNERIA ELETTRICA E
DELLE TECNOLOGIE DELL'INFORMAZIONE

MASTER DEGREE IN DATA SCIENCE

DATA MINING AND
MACHINE LEARNING PROJECT

Technical Report

Professors:

Prof.ssa Roberta Siciliano

Prof. Giuseppe Longo

Prof. Andrea Apicella

Students:

Lorenzo Bove

Matr. D03000141

Souha Aouididi

Matr. D03000144

Academic Year 2024/2025

1 Introduction and Objective

Facial Expression Recognition (FER) constitutes a pivotal domain within computer vision and artificial intelligence. The ability to automatically interpret human emotional states from visual data has far-reaching applications, ranging from neuromarketing and entertainment to psychological monitoring and sociable robotics. However, developing robust FER systems remains a challenging task due to the high inter-class similarity of facial micro-expressions and the significant intra-class variability introduced by factors such as lighting conditions, head pose, and subject demographics.

The primary objective of this project is to design, implement, and evaluate machine learning models capable of accurately classifying human emotions from static facial images. Specifically, the system aims to distinguish among seven fundamental emotional categories: ***Surprise, Fear, Disgust, Happy, Sad, Angry, and Neutral***. To ensure that the analysis reflects real-world conditions, the project utilizes the **Real-world Affective Faces Database (RAF-DB)**, a dataset characterized by its diversity and unconstrained environments.

To address this classification problem, this study adopts a comparative approach contrasting two distinct methodological paradigms.

- **Classical Machine Learning Approach:** This approach has been followed for the SVM model. This method relies on manual feature engineering, since the SVM is not as powerful as the models based on Neural Networks. In this project, multiple kernels for the SVM model have been compared in order to show how the accuracy changes.
- **Deep Learning Approach:** This approach has been followed for a custom-built Convolutional Neural Network (CNN) trained from scratch. This architecture is designed to automatically learn hierarchical feature representations directly from raw pixel data, capturing both low-level edges and high-level semantic facial structures without explicit manual intervention.

By benchmarking these two architectures, **this report aims to quantify the performance gap between traditional machine learning and modern deep learning techniques**, providing a detailed analysis of their respective accuracy, generalization capabilities, and suitability for the task of emotion recognition.

2 Dataset Explanation

The dataset utilized in this project is the **Real-world Affective Faces Database (RAF-DB)**. It is a dataset for facial expressions. The version used Contains more than 15000 RGB cropped facial images, tagged with basic or compound expressions by 40 independent taggers. The images in this database are of great variability in subjects' age, gender and ethnicity, head poses, lighting conditions, occlusions (e.g. glasses, facial hair or self-occlusion), post-processing operations (e.g. various filters and special effects), which makes this a real-world dataset. There are 7 different classes in which an image could be classified: ***Surprise, Fear, Disgust, Happy, Sad, Angry, and Neutral***. Each of these emotion has an identification number:

- 1: "surprise",
- 2: "fear",
- 3: "disgust",
- 4: "happy",
- 5: "sad",
- 6: "angry",
- 7: "neutral"

2.1 Data Partitioning

The images have been partitioned as follows:

- **Training Set:** 9,816 images.
- **Validation Set:** 2,455 images (created through an 80/20 stratified split of the original training data to maintain class distribution).
- **Test Set:** 3,068 images.

Each image has a label representing one of the 7 possible classes.



Figure 1: Example of image for each class

It is possible to see from the images above that these are cropped around the face to reduce background noise, thereby allowing the models to focus on relevant facial features and making interpretation easier.

3 Exploratory Data Analysis (EDA)

3.1 Missing and Duplicated Data

The first thing checked during the EDA is the possible presence of missing or duplicated data in the dataset. Upon inspection, the analysis revealed that the dataset is complete, with no missing values or duplicated entries detected.

3.2 Dimension and Format of the images

The next aspects that have been checked are the dimension of the images and their format. Ensuring uniformity across the dataset is crucial for the subsequent modeling phases, as inconsistent input sizes can lead to errors during batch processing. The inspection confirmed that the dataset is perfectly homogeneous: specifically, all the images have a fixed dimension of **100 x 100 pixels** and they all are stored in a **JPEG** format.

3.3 Distribution of the classes in Train, Validation and Test Set

A critical step in the EDA process involves analyzing the distribution of the target variable **"emotion"** across the 7 classes. Evaluating class balance is fundamental, as significant disparities in the number of samples per category can cause the model to be biased towards the majority classes, leading to poor generalization performance. Therefore, the frequency of each class was carefully examined across the training, validation, and test sets to ensure they share a consistent distribution. The following are the pie charts showing the distribution of the 7 classes respective for train, validation and test set:

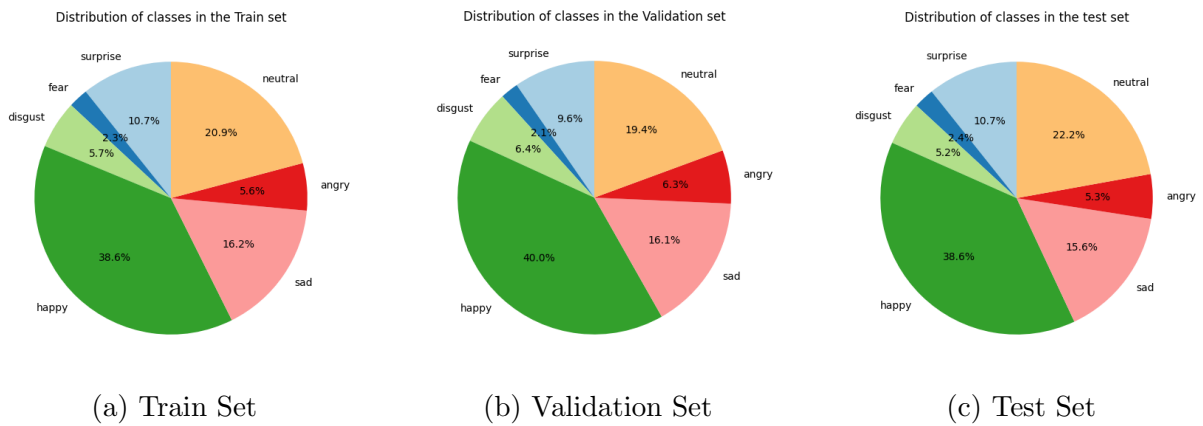


Figure 2: Distribution of the 7 classes across the Train, Validation, and Test sets.

It is possible to see that the distribution is similar in the three sets. However, there is a problem of unbalanced classes that needs to be fixed in future steps. Indeed, the class **"happy"** represents 38%-40% of the observations, while classes as **"fear"** or **"disgust"** represent only 2%-5% of the observations. This could cause that the model predicts correctly most of the observation in the class **"happy"** (just because it has seen more observations from that class) and obtains low results for the classes as **"fear"**.

4 SVM Model

The first model that has been applied in the project is the **Support Vector Machine (SVM)** model.

4.1 Theoretical Background

The SVM is a powerful, non-probabilistic supervised learning model used for classification and regression. The fundamental principle of an SVM is to find the optimal separating hyperplane that maximizes the margin between the different classes in the feature space. The SVM is based on a simpler model called **Maximal Margin Classifier**, which uses a linear hyperplane as decision boundary to classify the observations in one of the 2 classes, since it is used for binary classification by default. The Maximal Margin Classifier can be used only when the data are linearly separable, so each observation is correctly classified by the linear hyperplane. If this condition is met then, there are infinite possible hyperplane that divide the observations correctly. To choose the best one, it is necessary to compute the **margin**, which is the distance of each observation to the hyperplane, and find the hyperplane that maximize the margin, so the best hyperplane is the one that is the most distant from the observations. The points that are closer to the hyperplane, so the points that have the lowest value of the margin, are called **support vectors** because the hyperplane depends only on these points. So if one of the support vectors change position, also the hyperplane will change its slope. However, as said before, the Maximal Margin Classifier is usable only if the data are linearly separable, so in order to make the model more flexible, it is possible to use the Support Vector Classifier, which is similar to the Maximal Margin Classifier, but it has a hyperparameter (usually called C) that based on its value it allows the model to more flexible with wrong classified observations. In particular, the higher is the value of C and the more "severe" is the model, leading to a very strict margin. On the other hand, if the value of C is lower, then the margin is wider and the model is more tollerant to some misclassified observations. However, the SVC, like the MMC uses only a linear decision boundary. To solve this problem it is used the SVM, which can use different types of decision boundaries depending on the selected **kernel**. In this project have been compared 3 different SVM models with linear, polynomial and radial kernels.

4.2 Preprocessing Pipeline

The first phase that has been executed is the preprocessing of the images, in order to make the data usable by the SVM model. This phase is particularly important for classical Machine Learning model as the SVM, because they cannot process images by default, so it is required to make some transformations before.

The steps followed during the preprocessing are:

- **Training Set Rebalancing:** To address class imbalance, a **Random Oversampling technique** was applied **exclusively to the training data**. Minority classes were upsampled with replacement to match the majority class count, while validation and test sets remained unmodified to preserve real-world distributions.
- **Image Resizing:** Images were resized to spatial dimensions of 48×48 pixels to optimize computational efficiency.
- **Image Normalization:** Pixel intensity normalization was performed, rescaling values from $[0, 255]$ to the range $[0, 1]$.

- **Data Loading:** Given the nature of the image dataset, loading all samples simultaneously into the system memory would be inefficient and computationally expensive. To address this, we utilized Keras Data Generators (`ImageDataGenerator`). These tools establish a streaming pipeline that loads images from the disk in small batches only when needed ("lazy loading"), processing them on-the-fly. This approach ensures scalability and efficient memory management.
- **Data Generator Configuration:** Generators were instantiated with a batch size of 32 to ensure efficient memory usage during processing, and labels were encoded in a categorical (one-hot) format. Crucially, data shuffling was explicitly disabled (`shuffle=False`). This setting ensures a **deterministic processing order**, guaranteeing that the extracted feature vectors remain perfectly aligned with their corresponding labels and the original DataFrame indices—a necessary condition for correctly constructing the static input matrices (X and y) required by the SVM.
- **Batch Iteration:** A custom extraction loop iterates through the generator batch by batch. The `generator.reset()` command is invoked at the start to ensure the process begins correctly from the first sample.
- **Image Flattening:** Each batch of images, originally a 4D tensor with dimensions (32, 48, 48, 3), is reshaped into a 2D matrix. This operation "unrolls" the spatial dimensions, effectively treating every single pixel intensity as an independent feature. Consequently, each image is transformed into a flat feature vector of length 6,912 ($48 \times 48 \times 3$).
- **Label Decoding:** Since the generators yield labels in a one-hot encoded format (e.g., `[0, 0, 1, 0, ...]`), the `np.argmax` function is applied to convert them into single integer class indices (e.g., class 2). This step is necessary because standard SVM implementations typically require 1D target arrays rather than sparse matrices.
- **Data Aggregation:** Finally, the processed batches are accumulated and converted into static NumPy arrays (X and y). This procedure was executed separately for the training, validation, and test sets, resulting in the final input matrices ready for the SVM training phase.

4.3 Dimensionality Reduction: PCA

As said before, each pixel of the images is considered as an independent feature by the SVM, and in total there are 6,912 pixels. This can cause a problem called **overfitting**, because the number of variables inserted in the model is too high. Furthermore a lot of features are useless, so it is needed a reduction of dimensionality. To achieve this reduction **Principal Component Analysis (PCA)** was applied to train, validation and test set. In particular, the flattened image vectors ($1 \times 6,912$) were projected onto a lower-dimensional subspace retaining **100 components**. This drastic reduction (from 6,912 features after flattening to 100) preserves the most significant variations while discarding noise.

4.4 SVM with Linear kernel

The first SVM model used in the project is the SVM with a linear kernel, this means that the decision boundary used to classify each image is a line. The Linear Kernel is the simplest kernel function, defined as the dot product between two input vectors x and x' . It quantifies the similarity between samples directly in the original feature space without any mapping to higher dimensions. The formula is given by:

$$K(x, x') = \langle x, x' \rangle \quad (1)$$

This simplicity makes the Linear Kernel computationally efficient and less prone to overfitting, particularly when the number of features is large compared to the number of samples. This model, which establishes a baseline performance benchmark, was trained leveraging the lower-dimensional feature space obtained via PCA. Specifically, the `LinearSVC` implementation from *scikit-learn* was utilized, as it provides a faster and more efficient solver for large datasets compared to standard SVC implementations.

The model was configured with a regularization hyperparameter of $C = 1$, balancing the trade-off between margin maximization and training error. Additionally, the maximum number of iterations was explicitly increased to 10,000 (`max_iter=10000`) to ensure full convergence of the optimization algorithm on the projected data.

The model's performance was evaluated on both the validation and test sets. To gain deeper insights into the classification behavior, a **Normalized Confusion Matrix** was generated. This visualization allows for the assessment of per-class accuracy and the identification of specific confusion patterns between emotions (e.g., distinguishing between "Fear" and "Surprise"), independent of the number of samples in the test set. The resulting global accuracy on the test set was recorded at approximately 57%.

```
Accuracy (LinearSVC + PCA): 0.5752933507170795
```

```
Classification Report (LinearSVC + PCA):
```

	precision	recall	f1-score	support
angry	0.34	0.63	0.44	162
disgust	0.21	0.39	0.27	160
fear	0.18	0.51	0.26	74
happy	0.84	0.72	0.77	1185
neutral	0.66	0.45	0.54	680
sad	0.49	0.38	0.43	478
surprise	0.56	0.67	0.61	329
accuracy			0.58	3068
macro avg	0.47	0.54	0.48	3068
weighted avg	0.64	0.58	0.59	3068

Figure 3: Classification report of the 7 classes for the SVM with Linear kernel

From these metrics it is possible to see that the best classified class is "happy", indeed

both the precision (84%) and the recall (72%) are quite high. While on other classes, like "fear" or "disgust", the model does not perform well, leading to a F1-score of 26% and 27% respectively. To obtain further information it is convenient to check the Normalized Confusion Matrix.

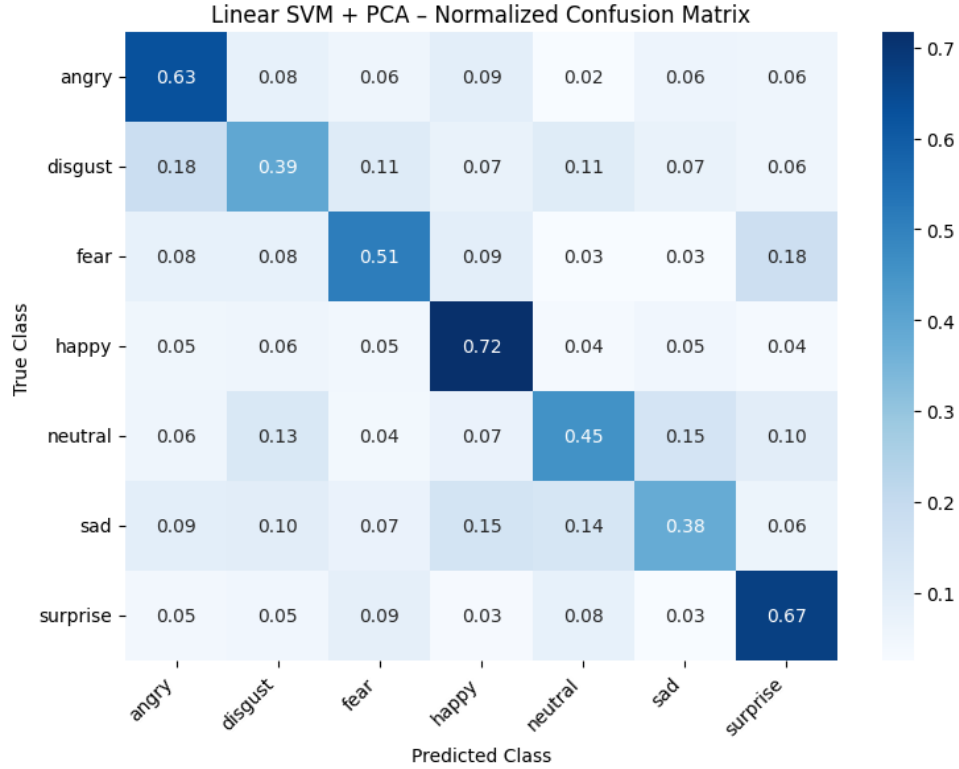


Figure 4: Normalized Confusion Matrix of the SVM with Linear kernel

From this confusion matrix it is possible to see that the model tends to misclassify images labeled as 'Fear' in 'Surprise', while 'Disgust' samples are frequently confused with 'Angry'. Overall, the limited accuracy yielded by the Linear Kernel SVM is an anticipated result. This performance gap is likely due to the inherent nature of SVMs, which treat pixels as independent features and lack the capacity to capture the complex spatial hierarchies typical of image data.

4.5 One vs Rest approach

Since the SVM is used for binary classification, it is needed a different approach to classify the observations in the case of multiclass classification. It is important to note that the function LinearSVC used in this project employs a **One-vs-Rest (OvR)** approach by default, distinguishing it from the standard One-vs-One (OvO) scheme used by non-linear SVMs (Polynomial and RBF). Consequently, the training process involved optimizing 7 distinct binary classifiers—one for each emotion class against all remaining classes. To

determine the final class prediction during inference, the model adopts a “Winner-Takes-All” decision rule: the system computes 7 distinct confidence scores (representing the signed distance from each classifier’s separating hyperplane) and assigns the final label by selecting the class associated with the highest numerical score, effectively applying an *argmax* function over the classifiers’ outputs.

4.6 SVM with Polynomial kernel

To address the limitations of the linear approach, a non-linear Support Vector Machine with a **Polynomial Kernel** was investigated. The polynomial kernel allows the model to learn non-linear decision boundaries by implicitly mapping the input vectors into a higher-dimensional feature space, where the data might become linearly separable. The kernel function $K(x, x')$ is defined as:

$$K(x, x') = (\gamma \langle x, x' \rangle + r)^d \quad (2)$$

where d represents the degree of the polynomial, γ is a scaling parameter, and r is an independent term (coefficient). This transformation enables the algorithm to capture interaction features between pixels (e.g., combinations of shapes) that a simple linear model cannot represent. However, training non-linear SVMs is computationally intensive and given the large size of the training dataset (approx. 29,000 images after balancing), training a polynomial SVM on the full set is prohibitive in terms of time.

To overcome this bottleneck, a **Stratified Subsampling** strategy was implemented for the training phase. The training set was reduced to a representative subset of $N = 5,000$ samples. Crucially, the `stratify` parameter was utilized during this reduction to ensure that the class distribution remained perfectly balanced, preserving the integrity of the prior oversampling step.

The model was instantiated with a polynomial **degree of 3** ($d = 3$) and a regularization parameter $C = 1$. It is important to note that while the model was trained on a reduced subset, the evaluation was performed on the **full validation and test sets** to ensure a robust and unbiased assessment of its generalization capabilities. The test accuracy obtained is lower than the test accuracy of the SVM with Linear kernel, indeed the SVM with Polynomial kernel reached an accuracy of approximately 52%.

Accuracy (Polynomial SVM + PCA): 0.5241199478487614				
Classification Report (Polynomial SVM + PCA):				
	precision	recall	f1-score	support
angry	0.34	0.30	0.32	162
disgust	0.13	0.38	0.19	160
fear	0.55	0.35	0.43	74
happy	0.73	0.76	0.75	1185
neutral	0.51	0.39	0.44	680
sad	0.39	0.37	0.38	478
surprise	0.61	0.39	0.48	329
accuracy			0.52	3068
macro avg	0.47	0.42	0.43	3068
weighted avg	0.56	0.52	0.53	3068

Figure 5: Classification report of the 7 classes for the SVM with Polynomial kernel

In general this model performs worst on every class except for the class "fear" where there is an improvement in the precision, leading to a better F1-score. To see how the general accuracy is lower, it is necessary to see the Normalized Confusion Matrix.

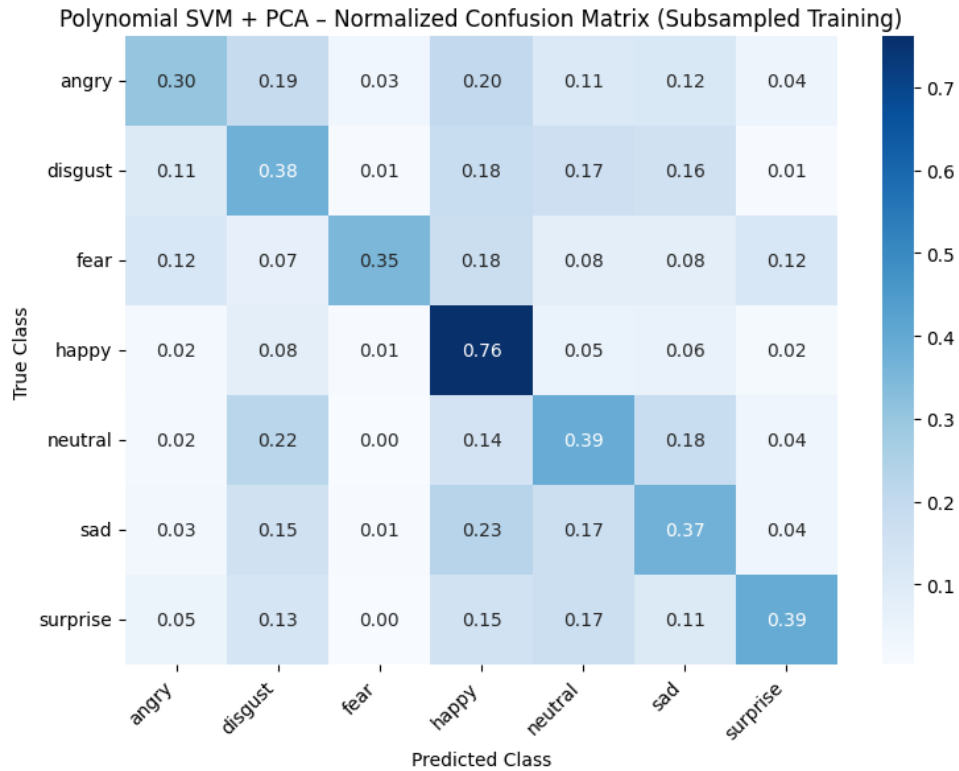


Figure 6: Normalized Confusion Matrix of the SVM with Polynomial kernel

The analysis highlights a distinct disparity in class performance. The 'Happy' class achieves the highest recall (0.76), indicating that the model is quite effective at identifying smiling faces. However, a significant bias is observed: the model frequently misclassifies other emotions as 'Happy' when uncertain. For instance, 23% of 'Sad' images and 20% of 'Angry' images are incorrectly predicted as 'Happy'. Conversely, the 'Angry' class proved the most challenging to classify (recall of 0.30), showing substantial confusion with 'Disgust' (0.19) and 'Happy' (0.20). This suggests that despite the non-linear kernel, the model still struggles to disentangle the subtle feature overlaps between negative emotions.

4.7 One-vs-One approach

Unlike the linear model which utilized a One-vs-Rest strategy, the non-linear SVC implementation (used here for the Polynomial kernel) adopts a **One-vs-One (OvO)** classification scheme. This approach constructs a dedicated binary classifier for every possible pair of classes. Given the 7 emotion categories, the model trained a total of:

$$\text{Classifiers} = \frac{N \times (N - 1)}{2} = \frac{7 \times 6}{2} = 21 \quad (3)$$

distinct classifiers (e.g., "Happy" vs. "Sad", "Angry" vs. "Fear"). This strategy is the standard for non-linear SVMs because training a kernel-based model on a smaller subset of data (only two classes at a time) is computationally more stable and efficient than training on the entire dataset simultaneously. During inference, a **"max-wins" voting mechanism** is employed: each of the 21 classifiers casts a vote for one of the two classes it distinguishes, and the class receiving the highest number of total votes is assigned as the final prediction.

4.8 SVM with RBF kernel

Finally, the Support Vector Machine was trained using the **Radial Basis Function (RBF)** kernel. The RBF kernel is widely considered the most versatile and powerful kernel for SVMs, as it is capable of handling highly non-linear decision boundaries by implicitly mapping the input space into an infinite-dimensional feature space.

Mathematically, the similarity between two feature vectors x and x' is calculated based on their Euclidean distance, weighted by a parameter γ (gamma). The kernel formula is defined as:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (4)$$

where:

- $\|x - x'\|^2$ represents the squared Euclidean distance between the two samples.
- γ is a kernel parameter that defines the "reach" or influence of a single training example.

Intuitively, the RBF kernel functions as a similarity measure: if two images have pixel values that are very close in the vector space, the distance $\|x - x'\|^2$ approaches zero, and

the kernel value approaches 1. Conversely, as the distance increases, the kernel value exponentially decays towards 0. This allows the model to create localized decision boundaries around clusters of similar emotions, potentially offering superior performance in distinguishing subtle facial expressions compared to the Linear and Polynomial approaches. Like the Polynomial model, this implementation utilizes the **One-vs-One (OvO)** strategy for multi-class classification. This model was trained on the full train set, reaching a test accuracy of 63%.

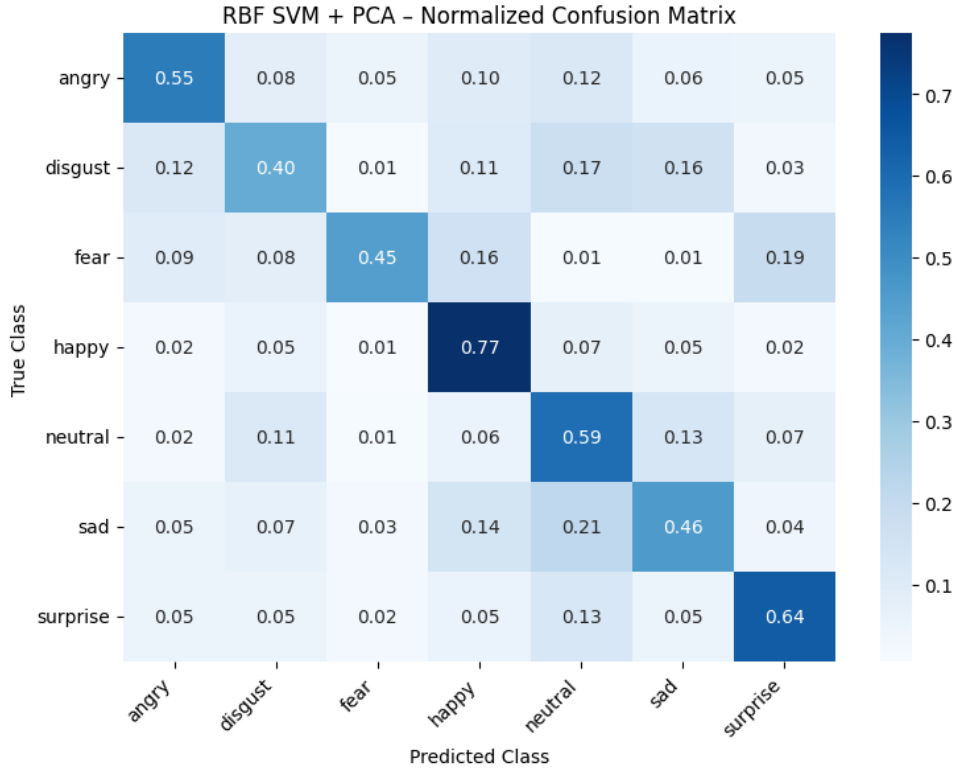


Figure 7: Normalized Confusion Matrix of the SVM with RBF kernel

The model exhibits a strong ability to identify positive and distinct expressions: the **Happy** class achieves the highest recall (0.77), followed by **Surprise** (0.64) and **Neutral** (0.59). However, the analysis highlights persistent challenges in distinguishing between subtle or visually similar negative emotions. Specifically, the **Disgust** category proved to be the most difficult to classify correctly (recall of 0.40), with significant confusion towards *Neutral* (0.17) and *Sad* (0.16). similarly, instances labeled as **Fear** (0.45) are frequently misclassified as *Surprise* (0.19), likely due to the shared visual features (e.g., widened eyes) between these two expressions. Despite these misclassifications, the diagonal dominance across most classes indicates that the RBF kernel successfully captures the non-linear decision boundaries required for this task.

4.8.1 Fuzzy Error Analysis and Decision Scores

4.8.2 Concept and Calculation

While Support Vector Machines are fundamentally binary classifiers that output a hard class label, they rely on a continuous internal metric known as the **Decision Function**.

This function calculates the signed distance of a sample x from the separating hyperplane defined by the model weights w and bias b .

These distances can be interpreted as **”Fuzzy Membership Scores”**: a higher positive value indicates not only class membership but also a higher degree of confidence, whereas a value close to zero implies uncertainty (the point is near the decision boundary). In a multi-class One-vs-Rest setting, the model computes 7 distinct distance scores (one for each emotion). The final prediction is simply the class with the maximum signed distance:

$$\hat{y} = \arg \max_{c \in \text{Classes}} (w_c^T \cdot x + b_c) \quad (5)$$

4.8.3 Analysis of Misclassified Sample (Index 2804)

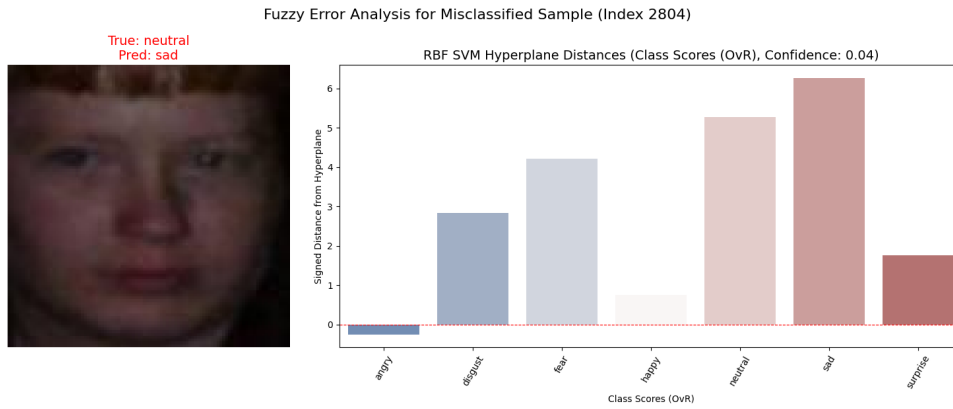


Figure 8: Fuzzy scores for the image n.2804 of SVM with RBF kernel

The plot illustrates a specific instance of misclassification analyzed through these fuzzy scores. The sample (Index 2804) has a **True Label of 'Neutral'**, but was incorrectly predicted as **'Sad'**.

The bar chart on the right visualizes the decision function output for all 7 classes. We can observe that the model was actually quite "confused" or "fuzzy" in its decision making:

- The score for **'Sad'** is the highest (≈ 6.2), triggering the false prediction.
- However, the score for the true class **'Neutral'** is also very high (≈ 5.3), indicating that the model did recognize significant neutral features in the face.
- Interestingly, the class **'Fear'** also shows a strong positive score (≈ 4.2), suggesting the model detected ambiguity in the facial expression (e.g., the eyes or mouth position) that overlapped with features typical of fear and sadness.

This analysis confirms that the error was not due to a complete failure of the model, but rather a "close call" where the decision boundary between 'Sad' and 'Neutral' was marginally crossed in the wrong direction.

4.8.4 ROC Curve

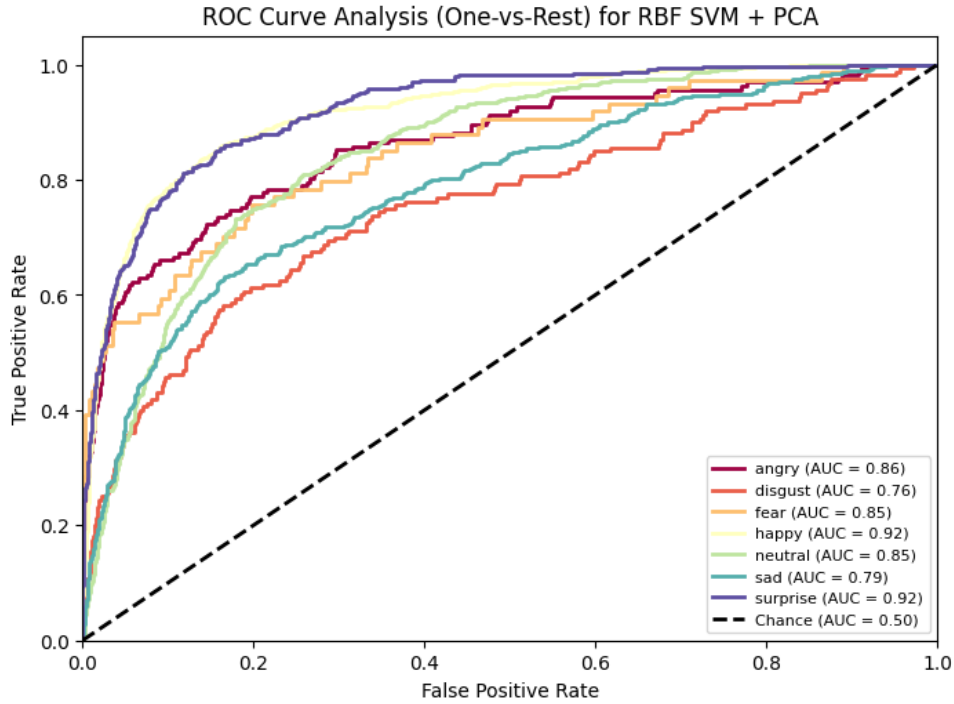


Figure 9: ROC Curve of the SVM with RBF kernel

The figure above illustrates the ROC curves obtained using the RBF Kernel. The Area Under the Curve (AUC) scores provide a quantitative measure of the model's separability for each emotion.

Consistent with previous findings, the model demonstrates excellent performance on distinct facial expressions: both 'Happy' and 'Surprise' achieve the highest AUC of 0.92, indicating a very high probability that the model will rank a randomly chosen positive instance higher than a negative one. Conversely, the curves for negative emotions are closer to the diagonal, reflecting the difficulty in classification. 'Disgust' yields the lowest performance with an AUC of 0.76, followed by 'Sad' (0.79). The remaining classes ('Angry', 'Fear', 'Neutral') show robust performance with AUC values clustered around 0.85-0.86, suggesting that while the RBF kernel improves general separation, the overlap in feature space for negative emotions remains a limiting factor.

5 Comparison of the different kernels

Until now this project has shown how the accuracy changes based on which kernel it has been chosen. To summarize what has been done, a bar chart of the test accuracies of the three models was created.

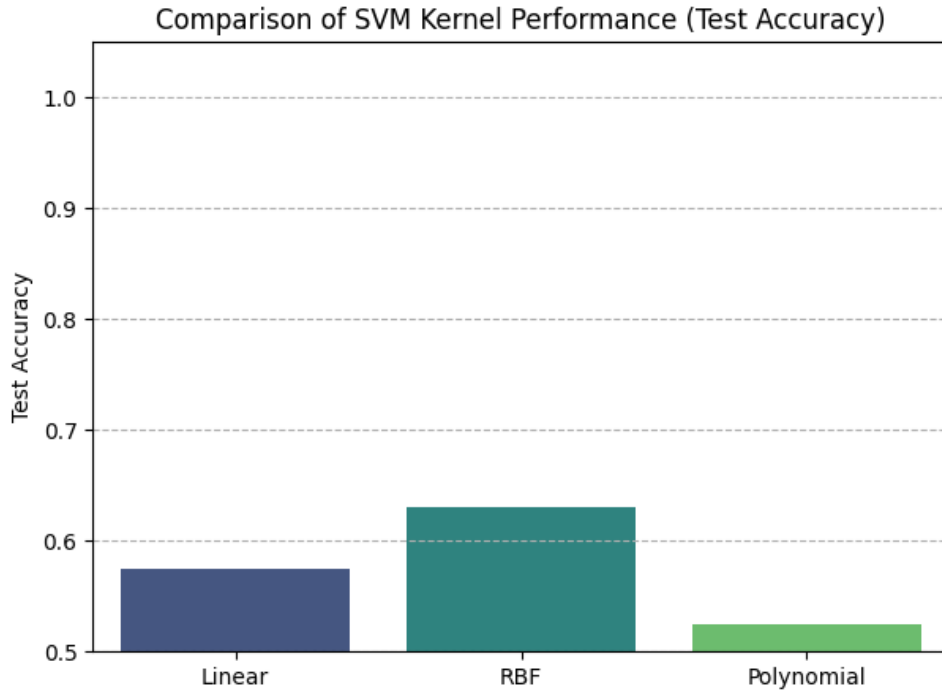


Figure 10: Bar chart of the test accuracies for the 3 SVM models

From the bar plot it is possible to see that the best model is represented by the SVM with RBF kernel (63% test accuracy), followed by the SVM with Linear kernel (57% test accuracy) and the SVM with Polynomial kernel (53% test accuracy).

6 Convolutional Neural Network (CNN)

6.1 Theoretical Background

Convolutional Neural Networks (CNNs) represent a specialized class of deep neural networks designed to process data with a grid-like topology, such as images. Unlike standard fully connected networks, which ignore the spatial structure of the input, CNNs are explicitly designed to recognize visual patterns directly from raw pixel data with minimal preprocessing.

The architecture of a CNN is constructed by stacking three main types of layers: Convolutional layers, Pooling layers, and Fully Connected layers. These components leverage three key architectural ideas to ensure shift, scale, and distortion invariance:

1. **Local Receptive Fields (Convolutional Layer):** In a traditional Feedforward Neural Network, every input pixel is connected to every hidden neuron, leading to an explosion of parameters for high-dimensional images. CNNs solve this by enforcing *sparse connectivity*. Each neuron in the hidden layer is connected only to a small, localized region of the input image, known as the **receptive field**.

The core operation is the **convolution**: a small matrix of learnable weights, called a **kernel** or **filter** (typically of size 3×3 or 5×5), slides over the input image. At each position, an element-wise multiplication is performed between the kernel and

the input pixels, followed by a summation. This process produces a scalar value that represents the activation of that specific neuron. By repeating this operation across the entire image, the network generates a 2D matrix called a **feature map**, which highlights specific visual features such as edges, curves, or textures.

2. **Parameter Sharing (Shared Weights):** A critical property of CNNs is that the parameters of the kernel are shared across all spatial locations of the input. This relies on the assumption that a feature useful in one part of the image (e.g., a vertical edge) is likely useful elsewhere.

Instead of learning a separate detector for every pixel, the same filter (with fixed weights) convolved across the entire input. This mechanism drastically reduces the number of trainable parameters compared to fully connected networks, making the model easier to train and less prone to overfitting. It also grants the network **translation invariance**: if an object moves in the image, its feature activation will simply shift in the feature map.

3. **Non-Linearity (Activation Function):** Following the linear convolution operation, the output is typically passed through a non-linear activation function. The most common choice in modern CNNs is the **Rectified Linear Unit (ReLU)**, defined as $f(x) = \max(0, x)$. This step introduces non-linearity into the network, allowing it to learn complex, high-dimensional boundaries and patterns that a purely linear model could not capture.
4. **Pooling (Subsampling):** Pooling layers are inserted between successive convolutional layers to progressively reduce the spatial dimensions (height and width) of the representation. The most common technique is **Max Pooling**, which selects the maximum value from a defined window (e.g., 2×2) in the feature map.

This operation serves two purposes: first, it reduces the computational load and the number of parameters for subsequent layers; second, it makes the features more robust to small variations and noise. By summarizing local neighborhoods, the network focuses on the *presence* of a feature rather than its exact pixel location.

5. **Fully Connected Layers (Classification Head):** After several convolutional and pooling blocks, the high-level reasoning is performed. The final 3D feature maps are flattened into a 1D vector and passed through one or more **Fully Connected (Dense) layers**. These layers function similarly to a standard Multi-Layer Perceptron (MLP), combining the extracted features to perform the final classification. The output layer typically uses a **Softmax** function to produce a probability distribution over the target classes (e.g., the 7 emotions).

6.2 Preprocessing Pipeline

The data preparation pipeline for the Convolutional Neural Network (CNN) involved several distinct stages designed to optimize the input for deep learning training.

1. **Class Balancing (Random Oversampling):** Consistent with the approach used for the SVM, the training set distribution was first analyzed. To address class

imbalance, a **Random Oversampling** technique was applied. The minority classes were upsampled with replacement until they matched the count of the majority class, ensuring that the model is exposed to a balanced distribution of emotions during training. The validation and test sets were left unmodified.

2. **Data Augmentation:** Unlike the SVM pipeline, training a CNN requires a robust strategy to prevent overfitting and improve generalization. To achieve this, real-time **Data Augmentation** was implemented using the `ImageDataGenerator`. Each training image was randomly transformed during each epoch using the following parameters:

- **Rescaling:** Pixel values were normalized to the range $[0, 1]$ (factor $1./255$).
- **Rotation:** Random rotations up to ± 25 degrees.
- **Shifts:** Random horizontal and vertical shifts up to 20% of the image dimension.
- **Zoom:** Random zoom adjustments up to 20%.
- **Horizontal Flip:** Random horizontal mirroring.

This process artificially expands the dataset and forces the network to learn features invariant to position, orientation, and scale.

3. **Generator Configuration:** Finally, data generators were instantiated to stream images in batches of **32**. Notably, the input target size was set to **100x100 pixels** (larger than the 48x48 used for SVM) to allow the CNN to capture finer spatial details.
 - For the **Training Generator**, shuffling was enabled (`shuffle=True`) to ensure stochasticity during gradient descent.
 - For the **Validation and Test Generators**, only pixel normalization (rescaling) was applied, and shuffling was disabled to maintain a fixed order for evaluation metrics.

6.3 Convolutional Neural Network Architecture

The proposed architecture is based on a Sequential Convolutional Neural Network (CNN), designed to process input images with dimensions of 100×100 pixels and 3 color channels. The model structure follows a pyramidal approach for feature extraction, progressively increasing the depth of the representation while reducing the spatial resolution.

The core of the architecture consists of four consecutive convolutional blocks. Each block operates according to a standardized sequence: a **Conv2D** layer with a 3×3 kernel for local feature extraction, followed by a **Batch Normalization** layer, a **ReLU** (Rectified Linear Unit) activation function, and finally a **MaxPooling2D** (2×2) layer for downsampling. The number of filters in the convolutional layers doubles progressively at each block, increasing from 32 to 64, 128, and finally 256, allowing the network to learn increasingly higher-level abstractions.

The final part of the network, dedicated to classification, begins with a **Flatten** layer that vectorizes the resulting 3D feature maps. This is followed by a dense (fully-connected)

layer of 512 neurons, which is also normalized and activated via ReLU. To mitigate the risk of overfitting, a **Dropout** layer with a rate of 0.3 was introduced, randomly deactivating 30% of the connections during the training phase. The final output is managed by a dense layer with 7 neurons and a **Softmax** activation function, which returns a probability distribution over the target classes.

6.4 Optimization and Training Stabilization

To ensure efficient learning, specific optimization strategies were adopted.

Batch Normalization The architecture makes extensive use of Batch Normalization after each convolution and in the first dense layer. This technique normalizes the output of the previous layer by subtracting the batch mean and dividing by the standard deviation. The primary objective is to counteract the phenomenon of *Internal Covariate Shift*, i.e., the variation in the distribution of layer inputs caused by parameter updates in preceding layers. Batch Normalization stabilizes the gradient, enabling the use of higher learning rates and reducing the dependence on weight initialization.

Adam Optimizer The model compilation utilizes the **Adam** (Adaptive Moment Estimation) optimization algorithm with a learning rate of $\eta = 0.0005$. Adam is a stochastic gradient descent method that computes adaptive learning rates for each parameter. The algorithm estimates the first (mean) and second (uncentered variance) moments of the gradients to adapt the learning rate. This characteristic makes Adam particularly effective in handling problems with sparse or noisy gradients, ensuring generally faster convergence compared to standard Stochastic Gradient Descent (SGD).

6.5 Training Configuration

- **Optimizer:** Adam (Adaptive Moment Estimation) with learning rate 0.0005.
- **Loss Function:** Categorical Crossentropy.
- **Epochs:** 50.
- **Callbacks:** Early Stopping and Learning Rate Reduction on plateau were implemented to optimize convergence.

6.6 Results of the CNN

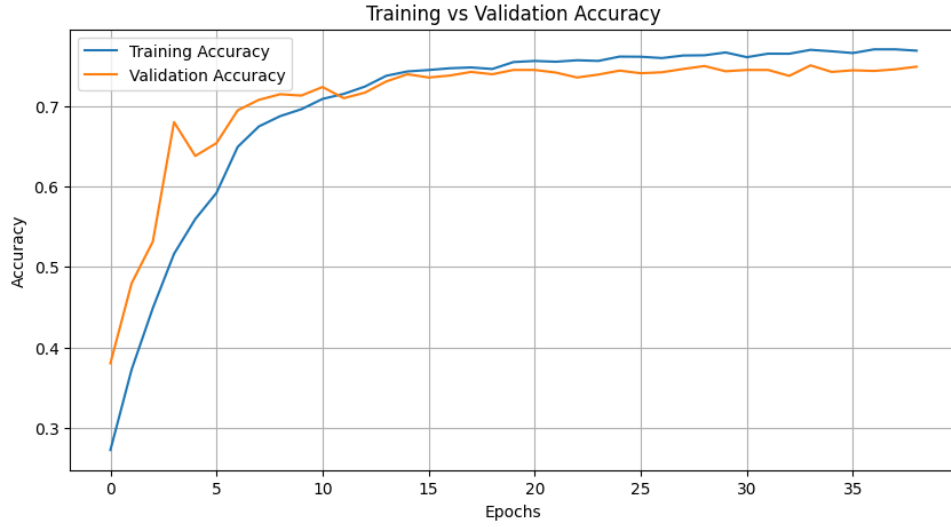


Figure 11: Train accuracy vs Validation accuracy for the CNN

The graph highlights a rapid increase in performance during the first 10 epochs, followed by a stabilization phase (plateau) around the 20th epoch. The model reaches a final accuracy of approximately 76% on the training set and 75% on the validation set.

A significant observation is the minimal gap between the two curves during the convergence phase: this indicates that the model demonstrates good generalization capabilities and does not suffer from significant *overfitting*. The proposed architecture, through the combined use of Batch Normalization and Dropout, successfully regularized the learning process.

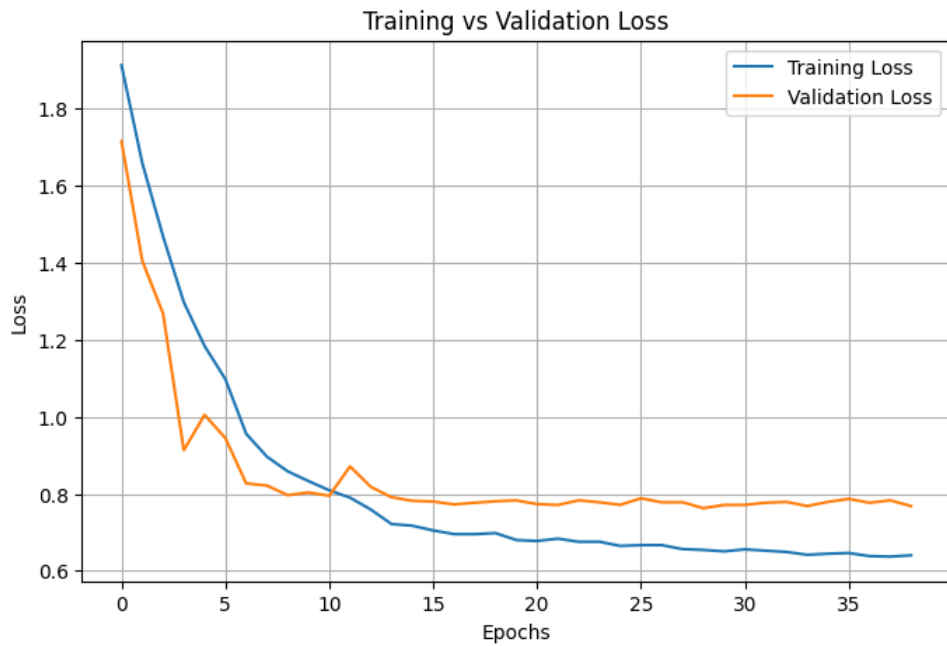


Figure 12: Train loss vs Validation loss

The graph shows a steep decline in the (*categorical crossentropy loss function*) values for both sets during the first 15 epochs, confirming that the network is effectively learning discriminative features.

From the 20th epoch onwards, a stabilization of the curves is observed. The training loss continues to decrease slightly, settling around a value of 0.65, while the validation loss plateaus near 0.80. Although a slight gap between the two curves is visible in the final phase (generalization gap), the asymptotic behavior of the validation loss suggests that the model is not degrading in performance (there is no drastic increase in validation error), thus maintaining good robustness.

Classification Report:				
	precision	recall	f1-score	support
angry	0.66	0.78	0.71	162
disgust	0.38	0.53	0.45	160
fear	0.44	0.59	0.51	74
happy	0.94	0.81	0.87	1185
neutral	0.74	0.69	0.71	680
sad	0.66	0.79	0.72	478
surprise	0.79	0.79	0.79	329
accuracy			0.76	3068
macro avg	0.66	0.71	0.68	3068
weighted avg	0.78	0.76	0.76	3068

Figure 13: Classification report of the 7 classes for the CNN

The table presents the detailed metrics (Precision, Recall, and F1-score) for each of the 7 emotion classes. The model achieves a **final test set accuracy of 76%**, a result consistent with the learning curves analyzed previously.

Analyzing individual classes reveals a direct correlation between the number of available samples (*support*) and model performance. The "happy" class, which is the most represented with 1185 samples, achieves the highest score with an F1-score of 0.87 and a precision of 94%.

Conversely, under-represented classes such as "disgust" (160 samples) and "fear" (74 samples) show lower performance, with F1-scores of 0.45 and 0.51, respectively. However, the *weighted average* F1-score stands at 0.76, confirming that, globally, the classifier maintains good reliability across the majority of tested instances.

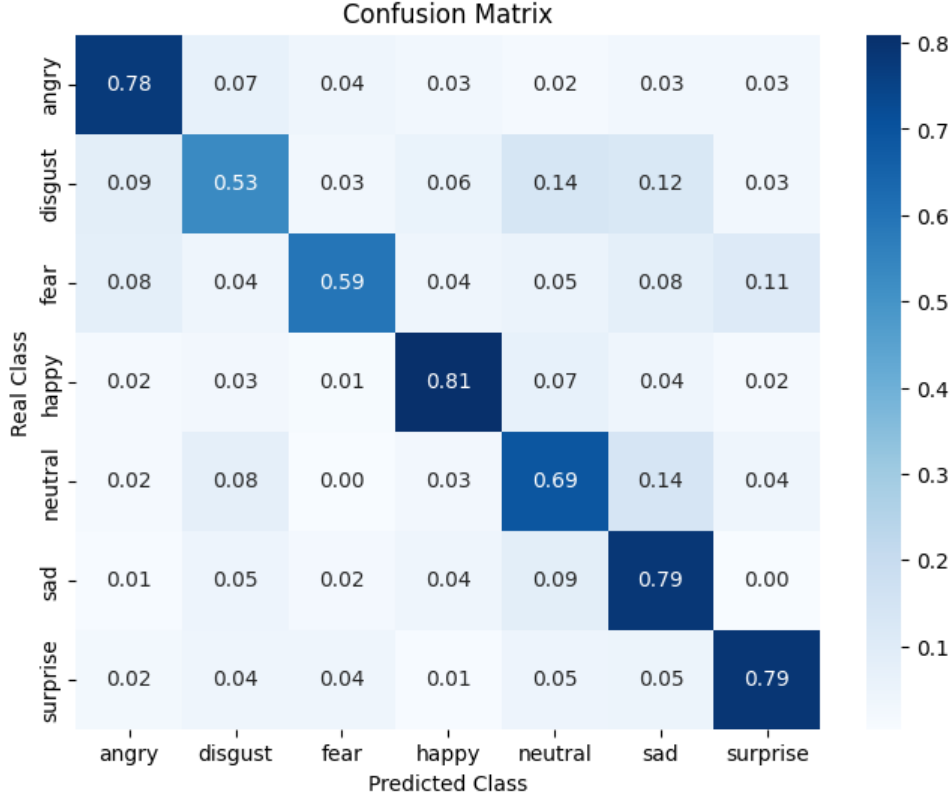


Figure 14: Normalized Confusion Matrix for the 7 emotion classes OF THE CNN

The normalized confusion matrix, shown in Figure 14, details the classification performance across the seven emotion categories. The diagonal elements represent the recall (True Positive Rate) for each class. The model demonstrates high proficiency in identifying the "happy" class, achieving an accuracy of 81%, followed closely by "surprise" and "sad" (both at 79%) and "angry" (78%).

However, the matrix highlights difficulties in correctly classifying under-represented classes. The "disgust" class records the lowest accuracy at 53%, with significant confusion towards "neutral" (14%) and "sad" (12%). Similarly, the "fear" class achieves only 59% accuracy, being frequently misclassified as "surprise" (11%) or "sad" (8%). A notable ambiguity is also observed between "neutral" and "sad" expressions, where 14% of neutral samples are incorrectly predicted as sad, suggesting a subtle overlap in the feature space learned by the CNN for these two states.

6.7 Saliency Map

To see how the CNN decide the class of an image it is useful to plot the **Saliency Map**. While standard backpropagation is typically used to update the network weights during training to minimize the loss function, Saliency generation employs a modification of this process for interpretability.

Given an input image I_0 , a class of interest c (the predicted emotion), and the class score function $S_c(I)$, the saliency map M is computed by calculating the gradient of the class score with respect to the input image pixel intensities:

$$M = \left| \frac{\partial S_c(I)}{\partial I} \right|_{I=I_0} \quad (6)$$

In this formulation, the network weights are frozen. The resulting gradient magnitude at each pixel (x, y) represents the sensitivity of the class score to infinitesimal changes in that pixel’s intensity. Consequently, pixels with high gradient magnitudes correspond to regions that the spatial hierarchy of the CNN deems most discriminative for the identified emotion.

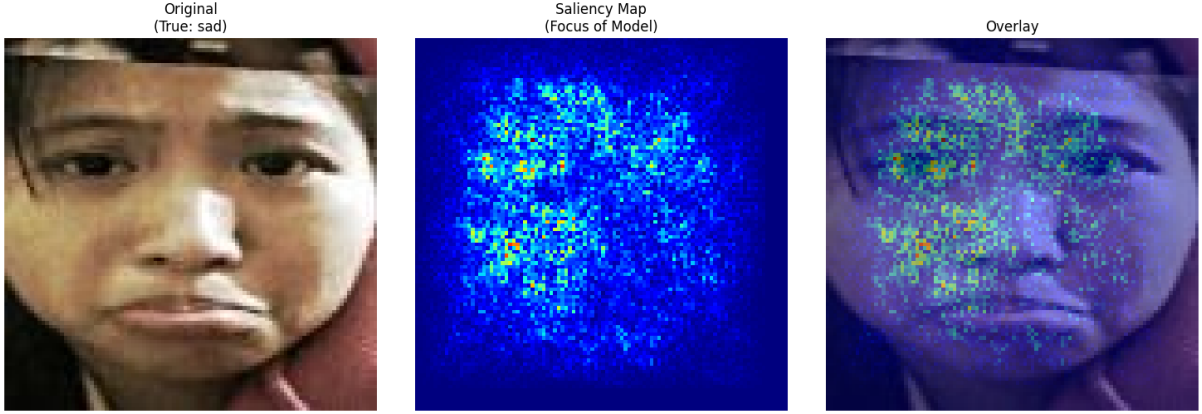


Figure 15: Saliency Map visualization for a sample classified as 'sad'. Left: Original image. Center: Saliency heatmap. Right: Overlay.

Figure 15 illustrates the application of this technique on a sample correctly classified as "sad". The heatmap (center) and the overlay (right) clearly show that the model’s focus is concentrated on key physiognomic areas. specifically the **eyes** and the **mouth curvature**. High-activation pixels (represented in warmer colors) correspond to the facial landmarks typically associated with the expression of sadness, such as the frowning of the lips and the shape of the eyes. The lack of significant activation in the background or on irrelevant details (like clothing) confirms that the CNN has effectively learned to extract discriminative facial patterns for emotion recognition.

7 Comparison CNN vs SVM

To validate the architectural choice of a Convolutional Neural Network, we compared its performance against traditional Machine Learning baselines based on Support Vector Machines (SVM) with different kernels: Linear, Polynomial, and Radial Basis Function (RBF).

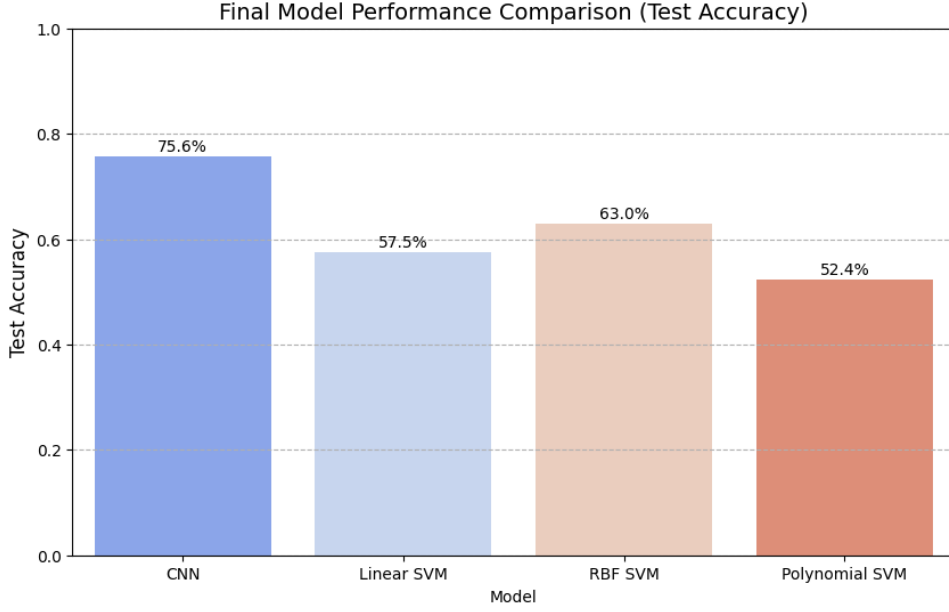


Figure 16: Final performance comparison (Test Accuracy) between the proposed CNN and SVM baselines.

Figure 16 illustrates the test accuracy obtained by each model. The proposed **CNN significantly outperforms all SVM variants**, achieving a top accuracy of **75.6%**. Among the traditional classifiers, the RBF SVM proved to be the most competitive, reaching 63.0%, yet still lagging behind the CNN by a substantial margin of over 12 percentage points. The Linear and Polynomial SVMs performed poorly (57.5% and 52.4% respectively).

8 Conclusion

This project successfully implemented and evaluated two distinct methodological approaches for Facial Expression Recognition (FER) using the Real-world Affective Faces Database (RAF-DB). The primary objective was to quantify the performance gap between classical machine learning techniques, represented by Support Vector Machines (SVM) with various kernels, and modern deep learning architectures, specifically a custom Convolutional Neural Network (CNN).

The experimental results provide a decisive answer to the research question. The comparative analysis demonstrated that the **Deep Learning approach significantly outperforms the classical methods**. The custom CNN achieved a test accuracy of **75.6%**, surpassing the best-performing SVM model (RBF kernel, 63.0%) by a substantial margin of over 12 percentage points. This performance gap highlights the limitations of traditional classifiers when dealing with high-dimensional raw pixel data. While SVMs rely on fixed or manually engineered feature spaces, the CNN proved capable of automatically learning complex, hierarchical representations—from simple edges to high-level semantic facial structures—essential for distinguishing subtle emotional nuances.

However, in order to achieve a better performance for the classification task seen during this project one idea could be to use **Transfer Learning**. In this way, using a pretrained

model based on CNN architecture trained on massive datasets, it is possible to leverage feature extractors that have already learned robust and complex visual patterns. This approach allows the model to start with a high level of semantic understanding, rather than learning everything from scratch, typically leading to higher accuracy and faster convergence even with limited training data.